

PRD-02: 系统架构与技术栈

版本: 1.0

日期: 2025-12

状态: 已实施

1. 概述

1.1 文档目的

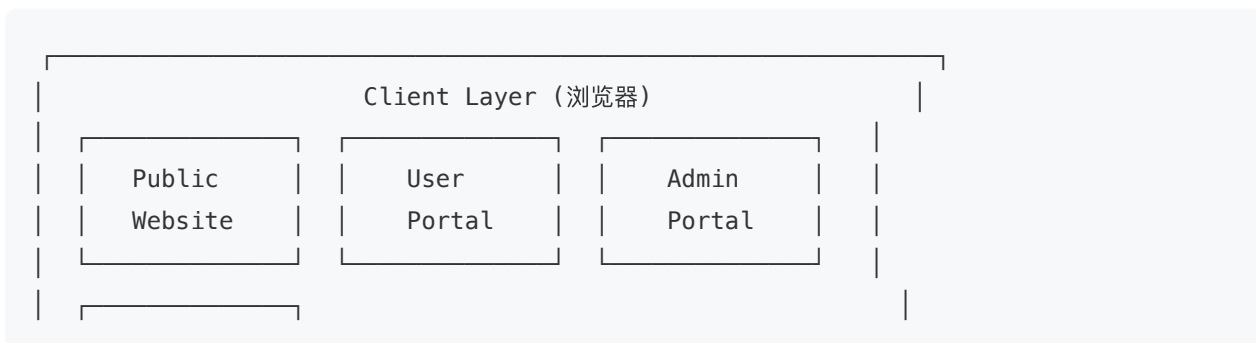
本文档描述 Blaze Robotics Academy 平台的系统架构设计和技术栈选择，为开发、部署和维护提供技术指导。

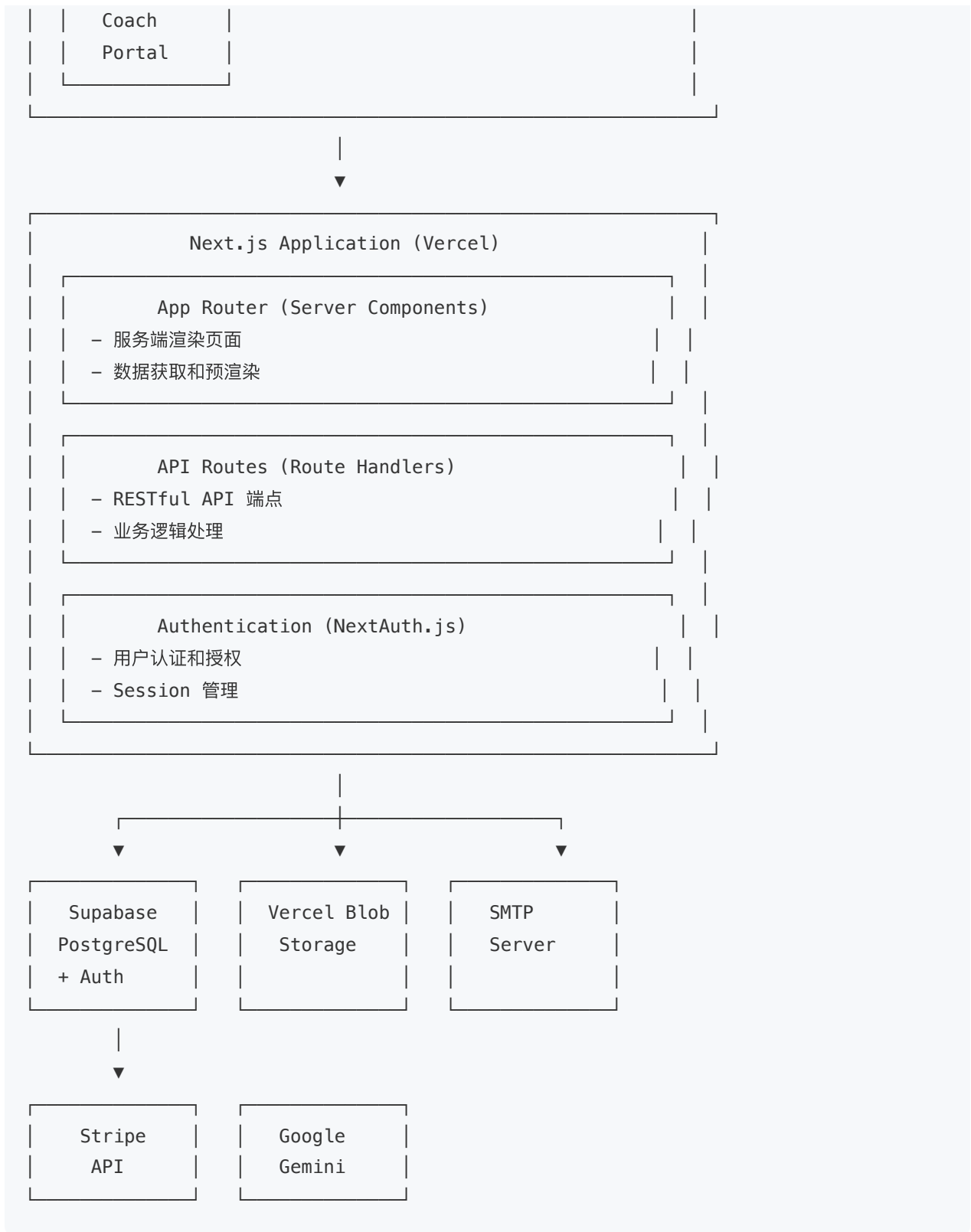
1.2 架构原则

- 可扩展性**: 支持多租户架构，易于扩展新校区
- 可维护性**: 使用现代化技术栈，代码结构清晰
- 安全性**: 多层安全防护，数据隔离
- 性能**: 优化加载速度，提升用户体验
- 可靠性**: 高可用性设计，错误处理完善

2. 整体架构

2.1 架构图





2.2 架构层次

2.2.1 表现层 (Presentation Layer)

- 技术：Next.js App Router、React 19、TypeScript

- 职责：
 - 用户界面渲染
 - 用户交互处理
 - 客户端状态管理
 - 路由管理

2.2.2 应用层 (Application Layer)

- 技术：Next.js API Routes、NextAuth.js
- 职责：
 - 业务逻辑处理
 - API 端点提供
 - 认证和授权
 - 数据验证

2.2.3 数据层 (Data Layer)

- 技术：Supabase (PostgreSQL)、Vercel Blob Storage
- 职责：
 - 数据存储和查询
 - 文件存储
 - 数据安全 (RLS)

2.2.4 集成层 (Integration Layer)

- 技术：Stripe API、Google Gemini API、SMTP
- 职责：
 - 第三方服务集成
 - 支付处理
 - AI 服务
 - 邮件发送

3. 多租户 (Franchise) 架构

3.1 架构模式

采用 单应用多租户 (Single Application Multi-Tenant) 模式：

- **单一代码库**：所有校区共享同一套代码
- **单一部署**：所有校区使用同一个部署实例
- **数据隔离**：通过 `franchise_id` 实现逻辑隔离
- **品牌定制**：通过 `branding_config` 实现个性化

3.2 数据隔离机制

3.2.1 Franchise 标识

- **路径模式**（当前实现）：
 - 总站： `/`
 - 子站： `/locations/{franchiseCode}`
 - 示例： `/locations/bellevue`
- **子域名模式**（未来支持）：
 - `bellevue.blazeroboticsacademy.org`
 - `issaquah.blazeroboticsacademy.org`

3.2.2 数据过滤

所有查询都基于 `franchise_id` 过滤：

- `course_instances.franchise_id`
- `course_enrollments.franchise_id`
- `course_locations.franchise_id`

3.3 品牌定制

通过 `franchises.branding_config` JSON 字段实现：

- Hero 标题和描述
- 联系信息
- 业务时间
- 特色内容

4. 技术栈

4.1 前端技术

技术	版本	用途
Next.js	16.0.7	React 框架, App Router
React	19.2.1	UI 库
TypeScript	5.x	类型系统
Tailwind CSS	3.4.17	样式框架
shadcn/ui	Latest	UI 组件库 (基于 Radix UI)
Lucide React	0.553.0	图标库

4.2 后端技术

技术	版本	用途
Next.js API Routes	16.0.7	API 端点
NextAuth.js	5.0.0-beta.30	认证和授权
Supabase	2.86.2	数据库和认证服务
PostgreSQL	(via Supabase)	关系型数据库

4.3 第三方服务

服务	用途
Stripe	支付处理
Google Gemini	AI 客服
Google OAuth	第三方登录

Vercel Blob	文件存储
SMTP	邮件发送

4.4 工具库

库	版本	用途
date-fns-tz	3.2.0	日期时间处理
rrule	2.8.1	重复规则处理
ical-generator	10.0.0	iCalendar 生成
bcryptjs	3.0.3	密码加密
nodemailer	7.0.11	邮件发送
@stripe/stripe-js	8.6.0	Stripe 前端 SDK
@stripe/react-stripe-js	5.4.1	Stripe React 组件
@ai-sdk/react	2.0.118	AI SDK React 组件

5. 数据库设计

5.1 数据库架构

使用 Supabase PostgreSQL，主要表结构：

5.1.1 核心业务表

- users：用户表
- franchises：Franchise 表
- course_locations：地点表
- courses：课程表
- course_categories：课程大类表

- `course_series` : 课程系列表
- `course_subcategories` : 子类标签表
- `course_assignments` : 课程分配表
- `course_instances` : 课程实例表
- `course_enrollments` : 报名表
- `learning_paths` : 学习路径表

5.1.2 关系表

- `course_subcategory_tags` : 课程-子类标签关系
- `course_prerequisites` : 课程先修条件
- `course_instance_coaches` : 实例-教练关系
- `learning_path_courses` : 学习路径-课程关系

5.1.3 系统表

- `teams` : 团队表
- `team_social_networks` : 团队社交媒体
- `password_reset_tokens` : 密码重置令牌
- `stripe_payment_events` : 支付事件审计

5.2 数据安全

5.2.1 Row Level Security (RLS)

- 所有表启用 RLS
- 用户只能访问自己的数据
- 管理员使用 `supabaseAdmin` (服务角色) 绕过 RLS

5.2.2 数据隔离

- 通过 `franchise_id` 实现多租户隔离
- 查询时自动过滤 `franchise_id`

6. 部署架构

6.1 部署平台

Vercel:

- 自动部署 (Git 集成)
- 全球 CDN
- 自动 HTTPS
- 环境变量管理

6.2 环境配置

6.2.1 开发环境

- 本地开发服务器
- 开发数据库 (Supabase)
- 测试 Stripe 密钥

6.2.2 生产环境

- Vercel 生产部署
- 生产数据库 (Supabase)
- 生产 Stripe 密钥
- 生产环境变量

6.3 CI/CD 流程

1. 代码提交 → GitHub
 2. 自动构建 → Vercel
 3. 自动部署 → 生产环境
 4. 环境变量 → Vercel Dashboard 配置
-

7. 安全架构

7.1 认证与授权

7.1.1 认证方式

- 邮箱/密码: 传统认证
- Google OAuth: 第三方登录

- **Session 管理**: JWT (NextAuth.js)

7.1.2 授权机制

- **角色基础访问控制 (RBAC)**:
 - `user`: 普通用户
 - `coach`: 教练
 - `admin`: 管理员
- **路由保护**: Middleware 检查
- **API 保护**: Session 验证

7.2 数据安全

7.2.1 密码安全

- bcryptjs 加密
- 密码重置令牌过期
- 邮箱验证令牌过期

7.2.2 支付安全

- Stripe Elements (PCI 合规)
- 不存储完整卡片信息
- Webhook 签名验证

7.2.3 输入验证

- TypeScript 类型检查
- API 参数验证
- SQL 注入防护 (参数化查询)

8. 性能优化

8.1 前端优化

- **服务端渲染 (SSR)**: Next.js App Router
- **静态生成 (SSG)**: 预渲染静态页面

- 代码分割：按路由自动分割
- 图片优化：Next.js Image 组件

8.2 后端优化

- 数据库索引：关键字段建立索引
- 查询优化：减少 N+1 查询
- 缓存策略：API 响应缓存（未来）

8.3 CDN 和缓存

- Vercel CDN：全球内容分发
 - 静态资源缓存：长期缓存
 - API 缓存：短期缓存（未来）
-

9. 监控与日志

9.1 错误监控

- Vercel Analytics：性能监控
- Console 日志：开发环境调试
- 错误追踪：未来集成 Sentry

9.2 日志记录

- 服务器日志：Vercel 日志
 - 数据库日志：Supabase 日志
 - 支付日志：Stripe Dashboard
-

10. 扩展性设计

10.1 水平扩展

- 无状态设计：API 无状态，易于扩展

- 数据库连接池：Supabase 自动管理
- CDN 分发：Vercel 自动处理

10.2 垂直扩展

- 数据库优化：索引和查询优化
 - 代码优化：减少不必要的计算
 - 缓存策略：减少数据库查询
-

11. 技术债务与改进计划

11.1 当前技术债务

- ⚠️ 部分 API 缺少错误处理
- ⚠️ 缺少单元测试
- ⚠️ 缺少 E2E 测试
- ⚠️ 部分组件需要重构

11.2 改进计划

- 📅 添加单元测试 (Jest)
 - 📅 添加 E2E 测试 (Playwright)
 - 📅 完善错误处理
 - 📅 性能监控集成
-

12. 附录

12.1 相关文档

- TOP_LEVEL_DESIGN.md : 顶层设计
- supabase-schema.sql : 数据库架构
- package.json : 依赖列表

12.2 技术参考

- [Next.js 文档](#)
- [Supabase 文档](#)
- [Stripe 文档](#)
- [NextAuth.js 文档](#)